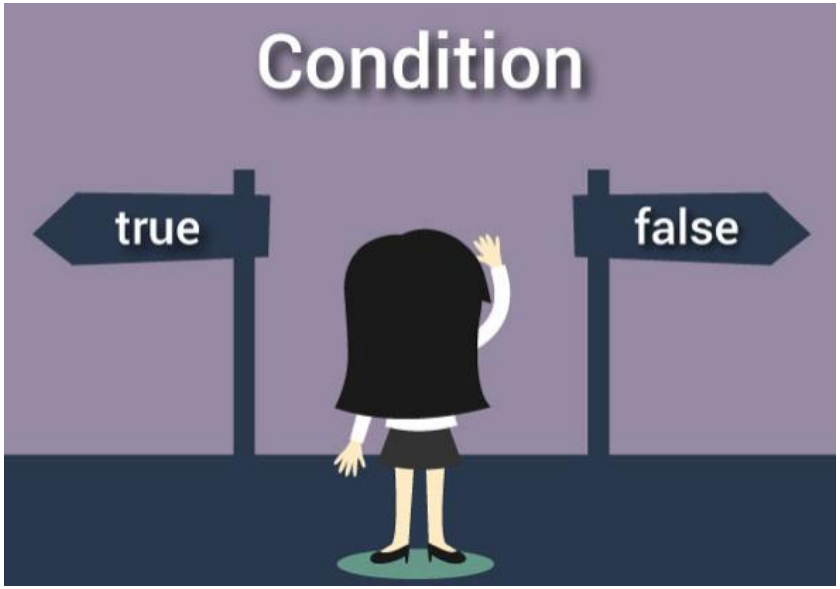
IS1201: Programming & Problem Solving

5. Conditions and Loops

Viraj Welgama

Conditions

- In programming, decision making is used to specify the order in which statements are executed.



C Ternary Operator (?:)

```
conditionalExpression ? expression1 : expression2
```

- The conditional operator works as follows:
  1. The first expression conditionalExpression is evaluated first. This expression evaluates to 1 if it's true and evaluates to 0 if it's false.
  2. If conditionalExpression is true, expression1 is evaluated.
  3. If conditionalExpression is false, expression2 is evaluated.

- A conditional operator is a ternary operator, that is, it works on 3 operands.

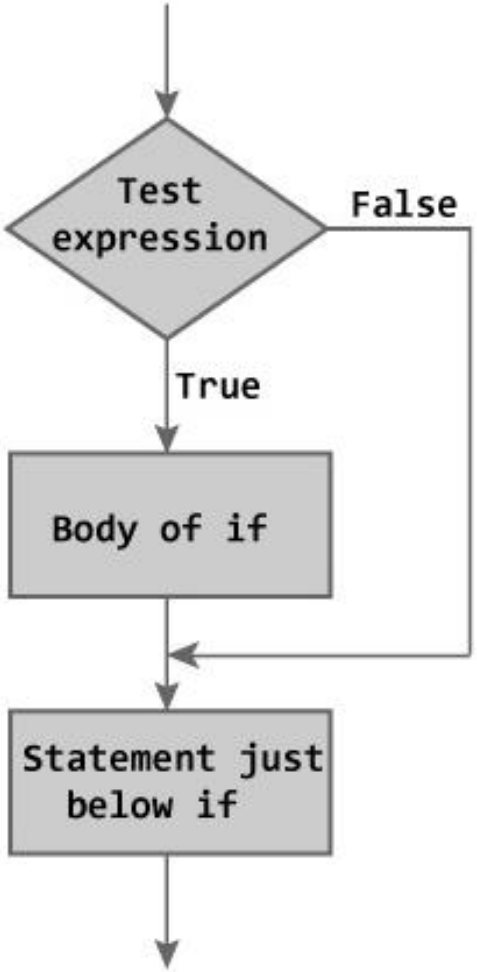
Example: Ternary Operator

```
int main() {

    int age;
    float dose;

    printf("\nEnter your age: ");
    scanf("%d", &age);

    (age>18) ? printf("You are an adult!\n") : printf("You are a kid!\n");

    dose = (age<12) ? 1.5 : 3;

    printf("your dose is %.1f", dose);

}
```

if Statement

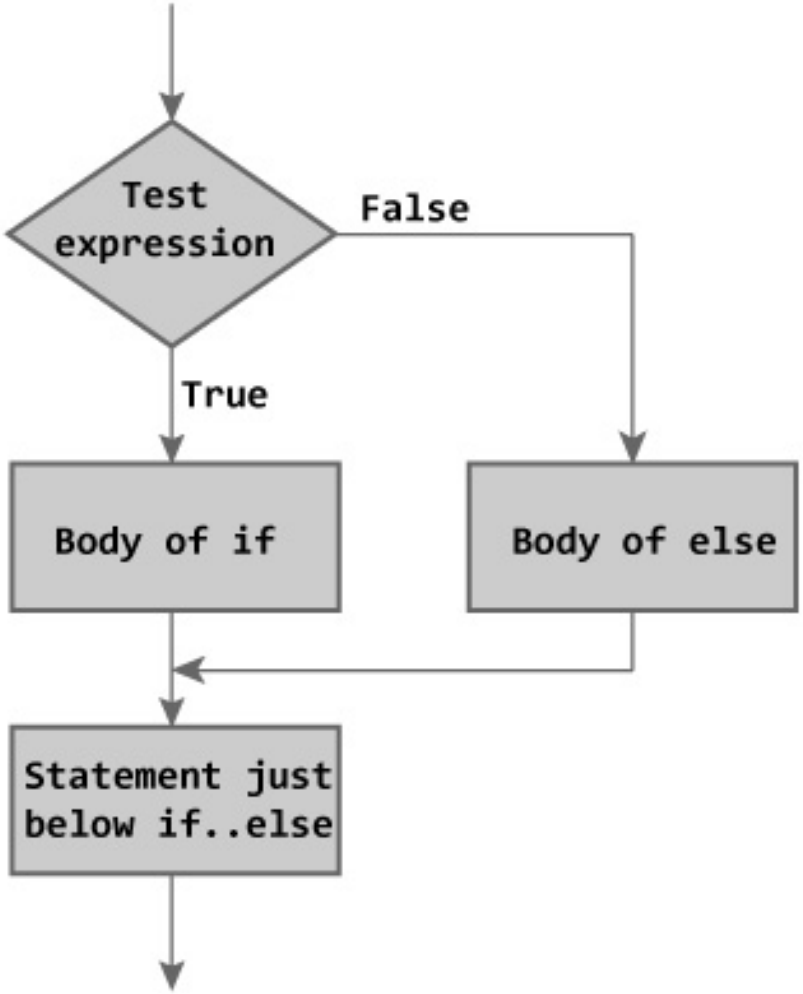
```
if(Expression) {
    //statements;
}
```

- The if statement evaluates the expression inside the parenthesis.
- If the test expression is evaluated to true (nonzero), statements inside the body of if is executed.
- If the test expression is evaluated to false (0) statements inside the body of if is skipped from execution.



if…else Statement

```
if(Expression) {
    //statements if true
}
else {
    //statements if false
}
```



- If test expression is true, codes inside the body of if statement is executed and, codes inside the body of else statement is skipped.
- If test expression is false, codes inside the body of else statement is executed and, codes inside the body of if statement is skipped.

Odd & Even Numbers

Odd & Even Numbers

```
int main() {

    int number;

    printf("Enter an integer: ");
    scanf("%d", &number);

    if(number%2==0) {
        printf("%d is a even number.", number);
    }
    else {
        printf("%d is an odd number.", number);
    }
}
```

Odd & Even Numbers

```
int main() {

    int number;

    printf("Enter an integer: ");
    scanf("%d", &number);

    if(number%2==0) {
        printf("%d is a even number.", number);
    }
    else {
        printf("%d is an odd number.", number);
    }
}
```

What if you need to print done! after printing the massage?

else if…

```
if(Expression1) {
    // statements to be executed when Expression1 is true
}
else if(Expression2) {
    // statements to be executed when Expression2 is true
}
else if(Expression3) {
    // statements to be executed when Expression3 is true
}
.
.
.
else {
    // statements to be executed otherwise
}
```

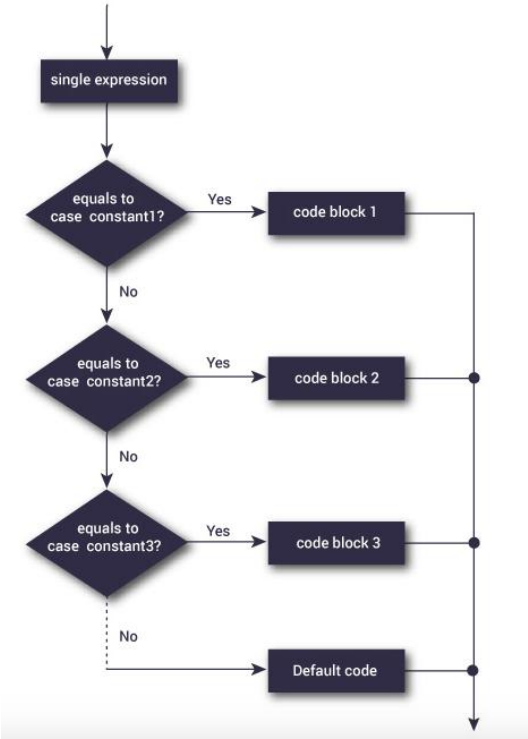
Example: Grades

- Write a program to give the grade when you enter your mark of a subject.
- The grade of the mark is defined as follows.
  - 00 – 24: E
  - 25 – 34: D
  - 35 – 49: C
  - 50 – 69: B
  - 70 – 100: A

Example: Grades

```
int main() {

    int mark;
    char grade;

    printf("Enter your mark: ");
    scanf("%d", &mark);

    if(mark>69) {
        grade = 'A';
    }
    else if(mark>49) {
            grade = 'B';
    }
    else if(mark>34) {
            grade = 'C';
    }
    else if(mark>24) {
        grade = 'D';
    }
    else {
        grade = 'E';
    }
    printf("your grade is %c", grade);
}
```

case Statement



- The if..else..if ladder allows you to execute a block code among many alternatives.
- If you are checking on the value of a single variable in if...else...if, it is better to use switch statement.
- The switch statement is often faster than multiple if...else.

Example:

```
int main () {

    char grade;

    printf("Enter your grade: ");
    scanf("%c", &grade);

    switch(grade) {
        case 'A' :
            printf("Excellent!\n" );
            break;
        case 'B' :
        case 'C' :
            printf("Well done\n" );
            break;
        case 'D' :
            printf("You passed\n" );
            break;
        case 'F' :
            printf("Better try again\n" );
            break;
        default :
            printf("Invalid grade\n" );
    }

    printf("Your grade is  %c\n", grade);

    return 0;
}
```

Rules of case statement

1. The **expression** used in a **switch** statement must have an integral or enumerated type, or be of a class type in which the class has a single conversion function to an integral or enumerated type.
2. You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
3. The **constant-expression** for a case must be the same data type as the variable in the switch, and it must be a constant or a literal.
4. When the variable being switched on is equal to a case, the statements following that case will execute until a **break** statement is reached.
5. When a **break** statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
6. Not every case needs to contain a **break**. If no **break** appears, the flow of control will fall through to subsequent cases until a break is reached.
7. A **switch** statement can have an optional **default** case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No **break** is needed in the default case.

Example: Grades

```c
int main() {

    int mark;
    char grade;

    printf("Enter your mark: ");
    scanf("%d", &mark);

    switch (mark) {
        case 0 ... 24:
            grade = 'E';
            break;
        case 25 ... 34:
            grade = 'D';
            break;
        case 35 ... 49:
            grade = 'C';
            break;
        case 50 ... 69:
            grade = 'B';
            break;
        case 70 ... 100:
            grade = 'A';
            break;
        default:
            grade = 'F';
    }
    printf("your grade is %c", grade);
}
```

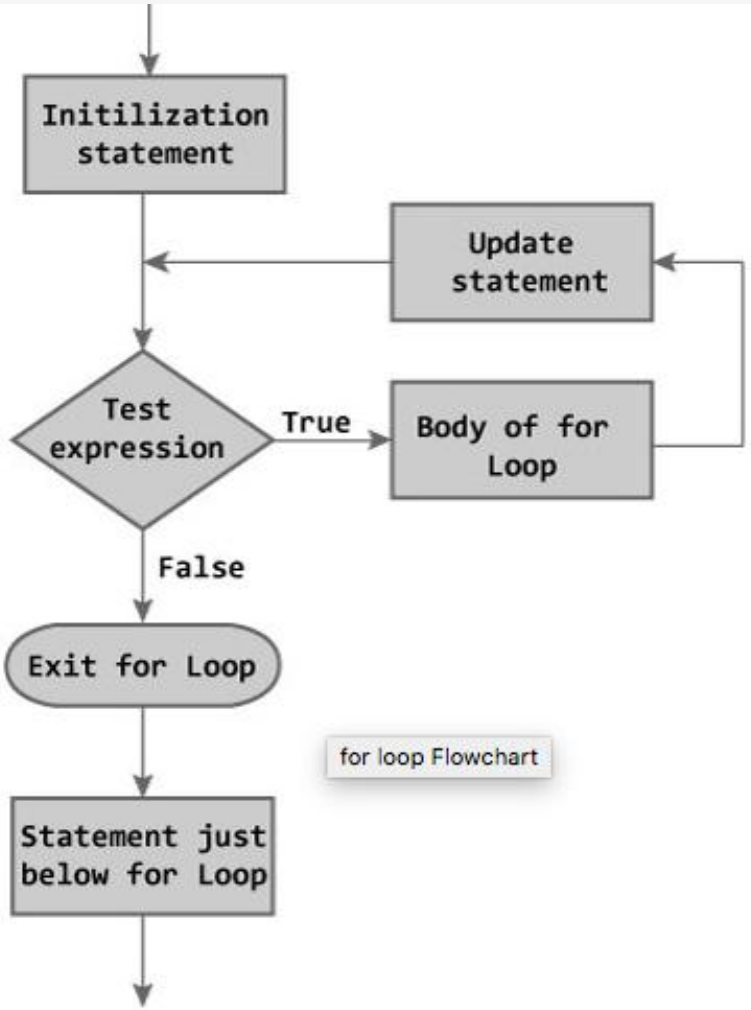
Loops

- Loops are used in programming to repeat a specific block of code.
- There are three loops in C programming:
  1. for loop
  2. while loop
  3. do...while loop

for Loop

```
for (initializationStatement; testExpression; updateStatement)
{
        // codes
}
```

1. The initialization statement is executed only once.
2. Then, the test expression is evaluated. If the test expression is false (0), the loop is terminated.
3. If the test expression is true (nonzero), codes inside the body of the loop is executed.
4. Then, the update statement is executed and update the variable.
5. This process repeats **until the test expression is false**.
6. The for loop is commonly used **when the number of iterations is known**.



for loop Flowchart

Example: Finding Factors

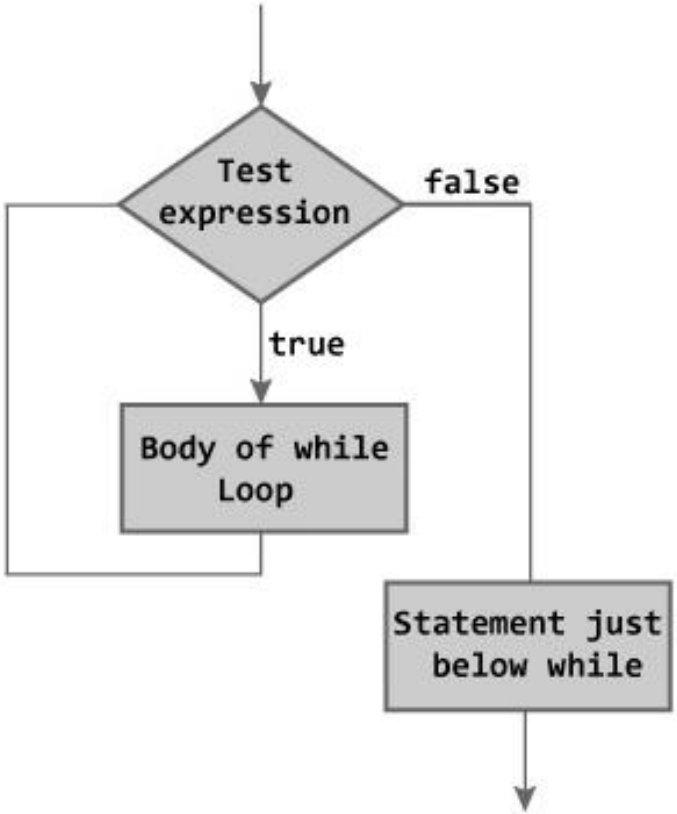
Example: Finding Factors

```
int main() {

    int number, i;

    printf("Enter a positive number: ");
    scanf("%d", & number);
    printf("Factors of %d are:\n", number);

    for (i = 1; i < number/2; ++i) {
        if(number%i==0) {
            printf("%d x %d = %d\n", i, number/i, number);
        }
    }

    printf("done!");

   return 0;
}
```

while Loop

```
while (testExpression)
{
    //codes
}
```

- The while loop evaluates the test expression.
- If the test expression is true (nonzero), codes inside the body of while loop are executed
- The test expression is evaluated again. The process goes on until the test expression is false.
- When the test expression is false, the while loop is terminated.



Example: while

- Finding factors of numbers till you feel enough!

Example: while

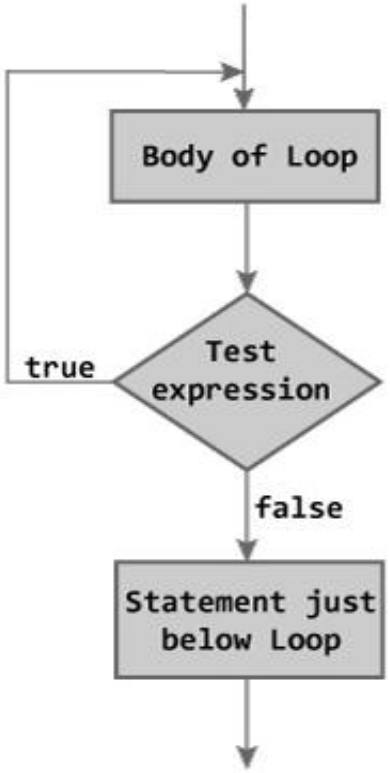
- Finding factors of numbers till you feel enough!

```
int main() {

    int number, i;

    while(number>-1) {

        printf("\nEnter a positive number: ");
        scanf("%d", & number);
        printf("Factors of %d are:\n", number);

        for (i = 1; i < number/2; ++i) {
            if(number%i==0) {
                printf("%d x %d = %d\n", i, number/i, number);
            }//if
        }//for
    }//while

    printf("exit the program...\n\n");

    return 0;
}
```

do...while Loop

```
do
{
    // codes
}
while (testExpression);
```

- The do...while loop is similar to the while loop with one important difference.
- The body of do...while loop is executed once, before checking the test expression.
- Hence, the **do...while loop is executed at least once.**



Nested Loops

- Nesting of loops is the feature in C that allows the looping of statements inside another loop.
- Any number of loops can be defined inside another loop, i.e., there is no restriction for defining any number of loops.
- The nesting level can be defined at n times.
- You can define any type of loop inside another loop;
  - for example, you can define **'while'** loop inside a **'for'** loop.

```
for (initialization; condition; update)
{
    for(initialization; condition; update)
    {
        // inner loop statements.
    }
    // outer loop statements.
}
```

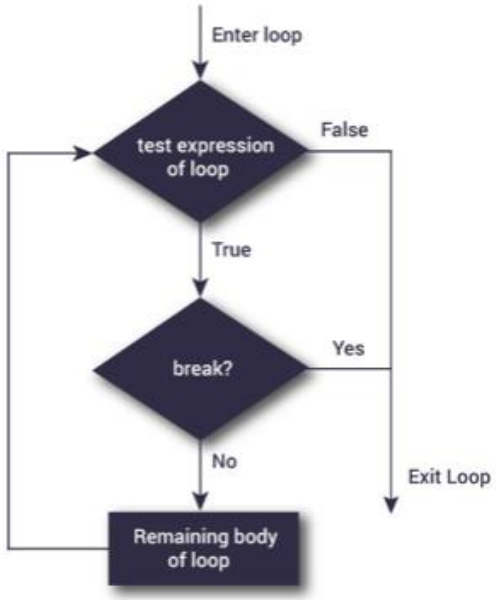
Example: Pyramids

- Write a program to print the following pattern for given number of rows form the character given by the user.
  - Ex: if user enters 5 as no of rows and '*' as the character, following pattern should be printed in the console.

```
*
**
***
****
*****
```

break

- The break statement terminates the loop (for, while and do...while loop) immediately when it is encountered.
- The break statement is used with decision making statement such as if...else.




```
while (test Expression)
{
    // codes
    if (condition for break)
    {
        break;
    }
    // codes
}
```

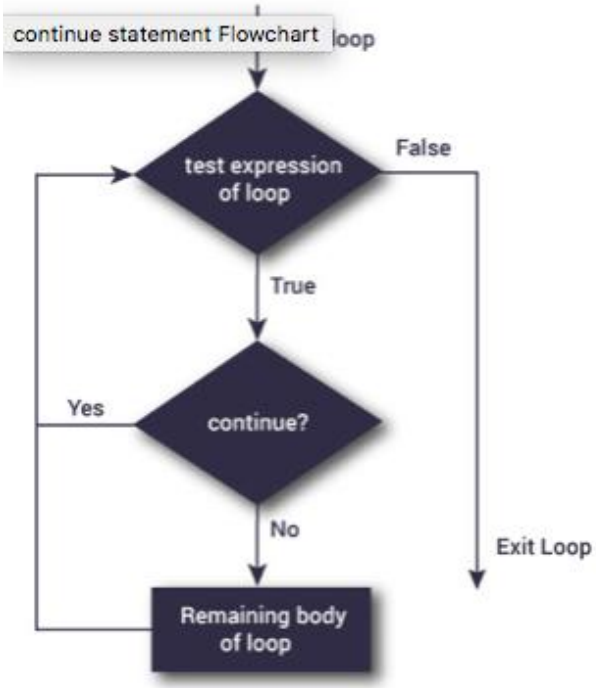
```
for (init, condition, update)
{
    // codes
    if (condition for break)
    {
        break;
    }
    // codes
}
```

Example: break

```
int main() {

    int num;

    while(1) {

        printf("\nEnter a number: ");
        scanf("%d", &num);

        if(num<0) break;
        if(num%2==0) {
            printf("%d is an even number.\n", num);
        }
        else {
            printf("%d is an odd number.\n", num);
        }
    }
    printf("Done!");
}
```

continue

- The continue statement skips some statements inside the loop.
- The continue statement is used with decision making statement such as if...else.




```
while (test Expression)
{
    // codes
    if (condition for continue)
    {
        continue;
    }
    // codes
}
```

```
for (init, condition, update)
{
    // codes
    if (condition for continue)
    {
        continue;
    }
    // codes
}
```

Example: continue

- Write a program to print numbers divided by 5, but not 10 between 1-100.

Example: continue

- Write a program to print numbers divided by 5, but not 10 between 1-100.

```
int main() {

    for(int i=0; i<=100; i++) {
        if(i%5==0) {
            if(i%10==0) continue;
            printf("%d\n", i);
        }
    }
    printf("Done!");
}
```

Example: continue

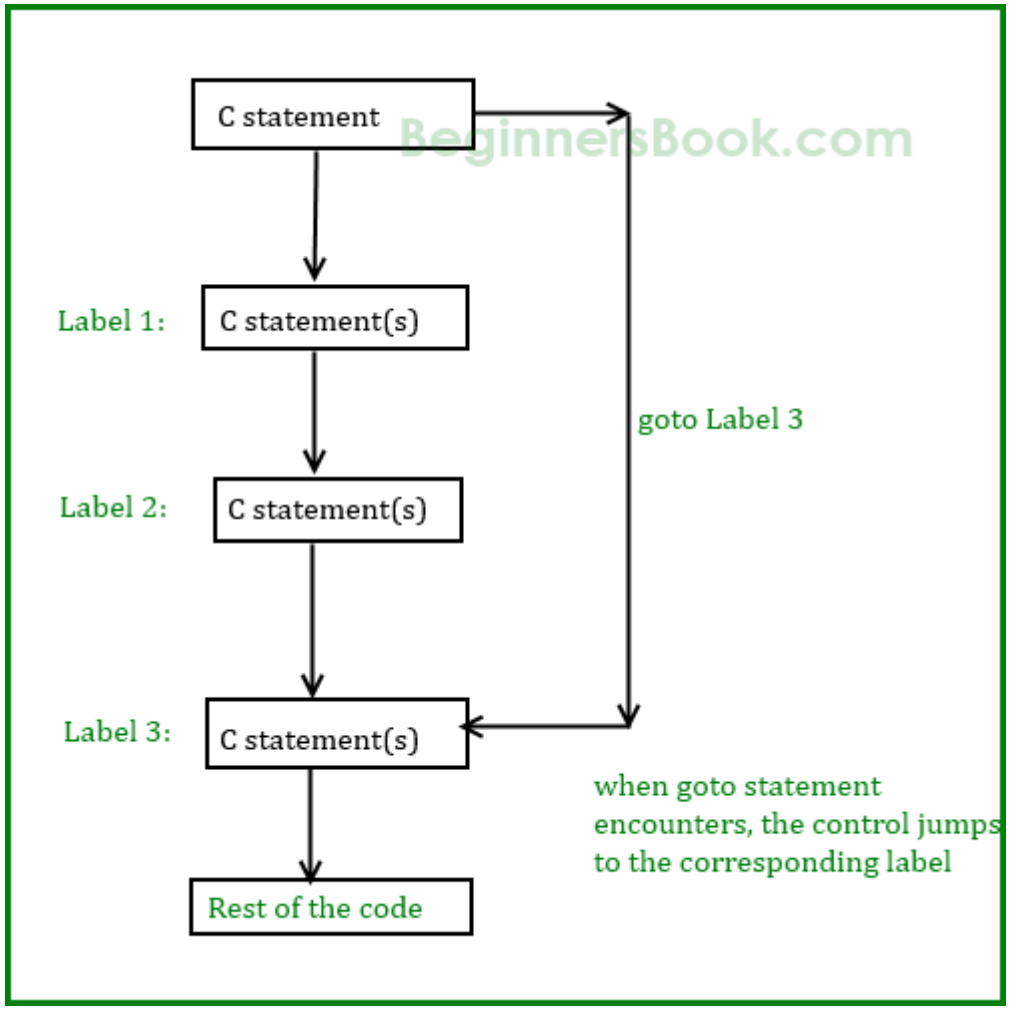
- Write a program to print numbers divided by 5, but not 10 between 1-100.

```
int main() {

    for(int i=0; i<=100; i++) {
        if(i%5==0) {
            if(i%10==0) continue;
            printf("%d\n", i);
        }
    }
    printf("Done!");
}
```

'for' loop initial declarations are only allowed in C99 or C11 mode

goto

- The goto statement is used to alter the normal sequence of a C program.
- When goto statement is encountered, control of the program jumps to label: and starts executing the code.
- However, The goto statement is rarely used because it makes program confusing, less readable and complex.
  - Also, when this is used, the control of the program won't be easy to trace, hence it makes testing and debugging difficult.



Example: goto

```
// Program to calculate the sum and average of positive numbers
// If the user enters a negative number, the sum and average are displayed.


int main() {

    const int maxInput = 100;
    int i;
    double number, average, sum = 0.0;

    for (i = 1; i <= maxInput; ++i) {
        printf("%d. Enter a number: ", i);
        scanf("%lf", &number);

        // go to jump if the user enters a negative number
        if (number < 0.0) {
            goto jump;
        }
        sum += number;
    }

jump:
    average = sum / (i - 1);
    printf("Sum = %.2f\n", sum);
    printf("Average = %.2f", average);

    return 0;
}
```

Problems

1. Write a program to take a positive integer (let say n) as an input from the user and calculate the sum of all integers up to n.

2. Write a program to determine the given number is prime number

3. Write a program to return the largest Integer from the given set of numbers.

4. Write a C Program to reverse a number